CT7201

Data Analysis Continued

This work is licensed under a Creative Commons Attribution-ShareAlike 3.0 Unported License.
It makes use of the works of Mateus Machado Luna.

Finding Correlations

- Continuing from where we left off on our machine learning project
- As the dataset in this case is not all that large, you can compute pearson's r, or the standard correlation coeffficient between every pair of attributes, using the "corr()" method

$$r = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_i (x_i - \overline{x})^2}\sqrt{\sum_i (y_i - \overline{y})^2}}$$

Finding Correlations

```
In [38]:  corr_matrix = housing.corr()
```

```
In [39]:  corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
Out[39]: median_house_value    1.000000
         median_income         0.687160
         total_rooms           0.135097
         housing_median_age    0.114110
         households            0.064506
         total_bedrooms        0.047689
         population           -0.026920
         longitude            -0.047432
         latitude             -0.142724
         Name: median_house_value, dtype: float64
```

What does this mean?

- These measurements are correlations against the median house value
- The correlation value ranges from -1 to 1, the closer it is to 1, the stronger the positive correlation
- The closer it is to -1, the stronger the negative correlation
- At the 0 point, there is no correlation

$r = ¿$

```
median_house_value    1.000000
median_income         0.687160
total_rooms           0.135097
housing_median_age    0.114110
households            0.064506
total_bedrooms        0.047689
population           -0.026920
longitude            -0.047432
latitude             -0.142724
Name: median_house_value, dtype: float64
```

What does this mean?

- Watch out, because this measure will only catch linear correlations
- i.e. “if X goes up, Y goes up”
- Or “if X goes up, Y goes down” $r = ¿$
- Or similar measures
- It will miss “If X goes up then Y goes up and then curves back down” or anything else non-linear or more complex

```
median_house_value    1.000000
median_income         0.687160
total_rooms           0.135097
housing_median_age    0.114110
households            0.064506
total_bedrooms        0.047689
population           -0.026920
longitude            -0.047432
latitude             -0.142724
Name: median_house_value, dtype: float64
```

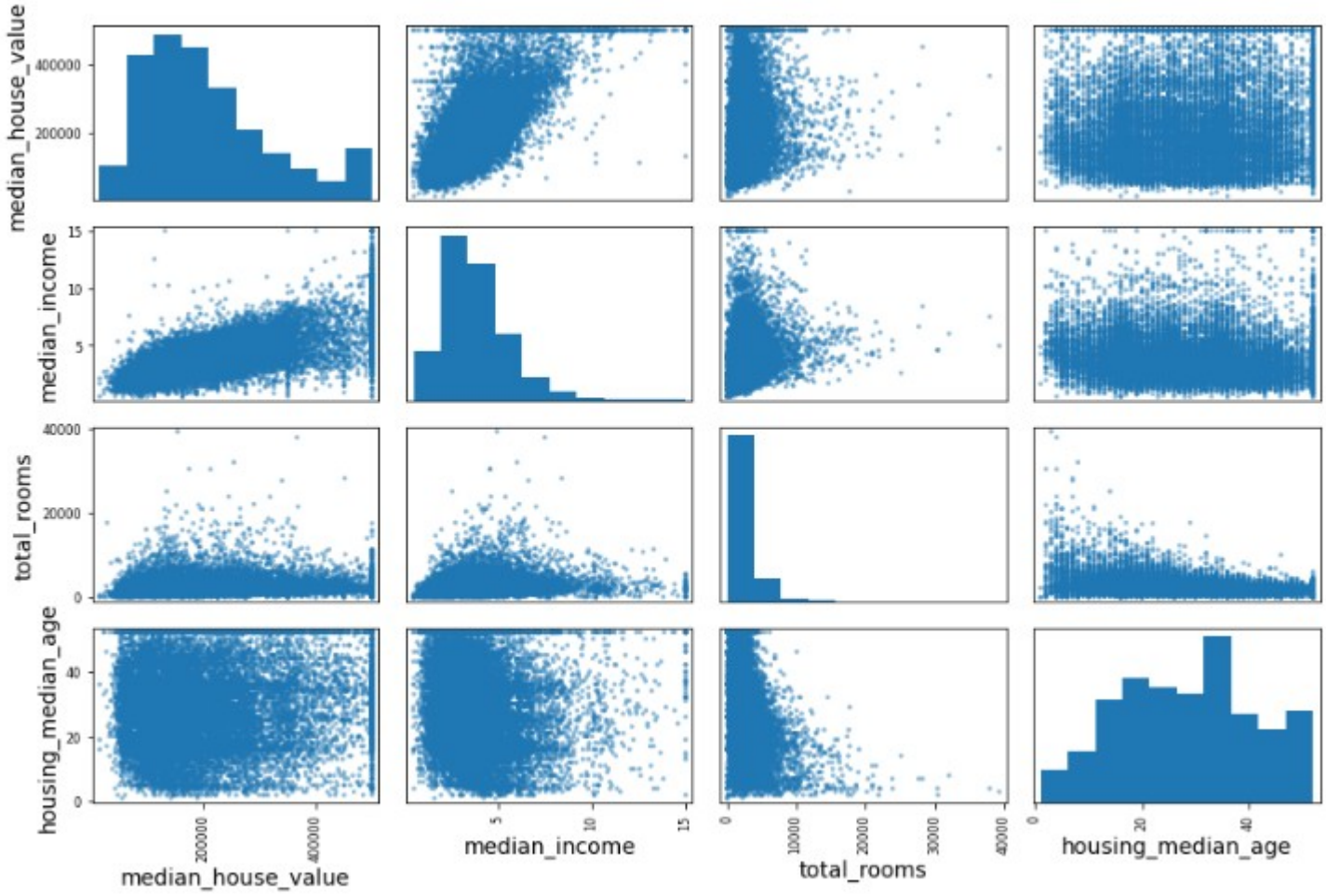
Scatter Matrix Plots

- Another approach to identify correlations, is to use a scatter matrix plot
- This plots every numerical attribute against every other numerical attribute
- With 11 numerical attributes, we would get 121 plots, which isn’t going to fit well on the page, so we can investigate just the attributes which we already know appear to be correlating well with median house value

Scatter Matrix Plots

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
save_fig("scatter_matrix_plot")
```


median_house_value
median_income
total_rooms
housing_median_age
400000
200000
15
10
5
40000
20000
0
40
20
0
200000
400000
5
10
15
0
10000
20000
30000
40000
0
20
40

Looking deeper

- The first thing to notice is the diagonal – if pandas plotted each variable against itself, you’d end up with a diagonal row of straight lines which isn’t all that helpful – so it displays a histogram of the attribute instead in those boxes
- Each section can then be analysed to identify how well it correlates with median house value
- The clearest correlation (as indicated by our earlier test also) is with median income

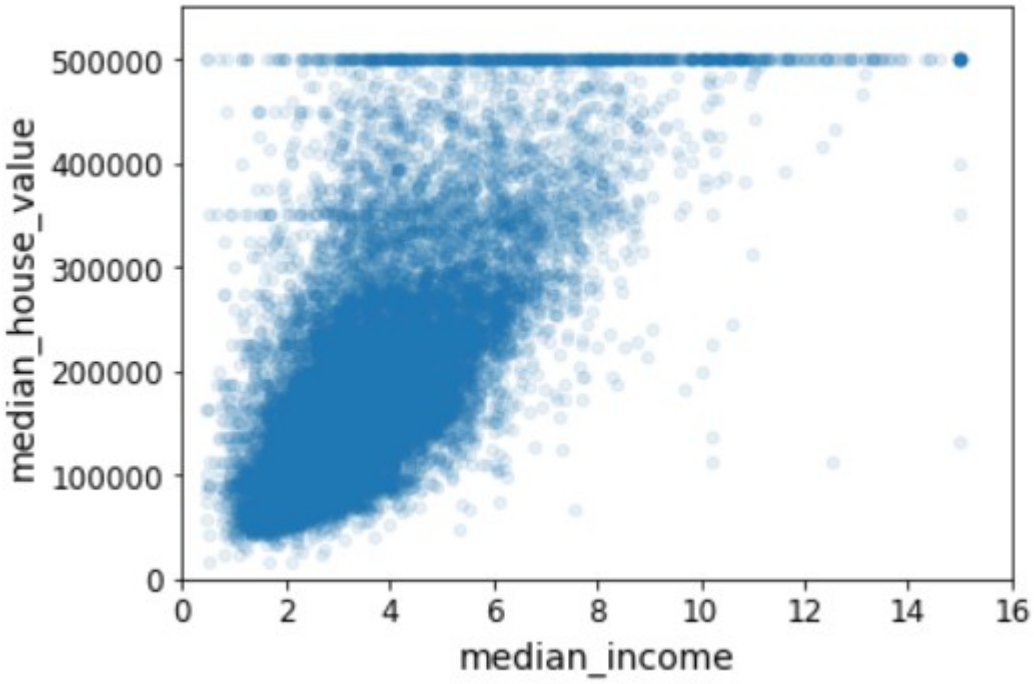
Looking deeper

- The first thing to notice is the diagonal – if pandas plotted each variable against itself, you’d end up with a diagonal row of straight lines which isn’t all that helpful – so it displays a histogram of the attribute instead in those boxes
- Each section can then be analysed to identify how well it correlates with median house value
- The clearest correlation (as indicated by our earlier test also) is with median income, so lets plot that

Looking deeper

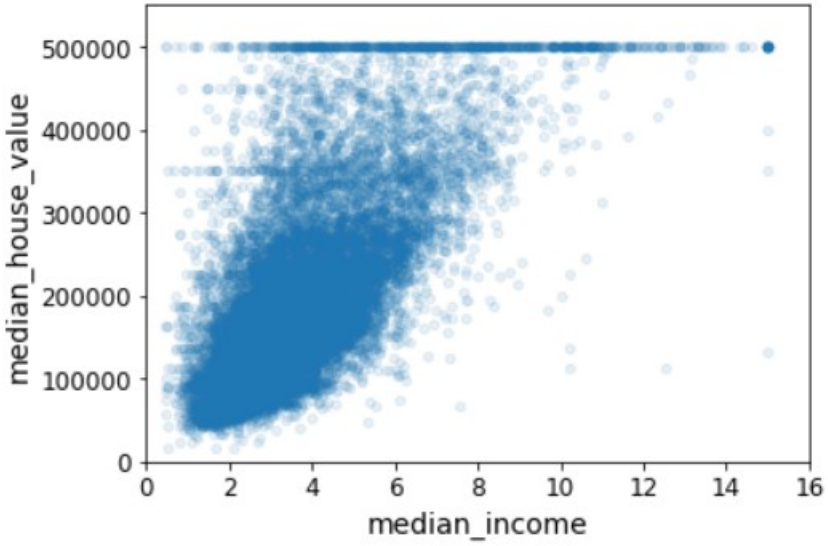
```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1)
plt.axis([0, 16, 0, 550000])
save_fig("income_vs_house_value_scatterplot")
```

Looking deeper



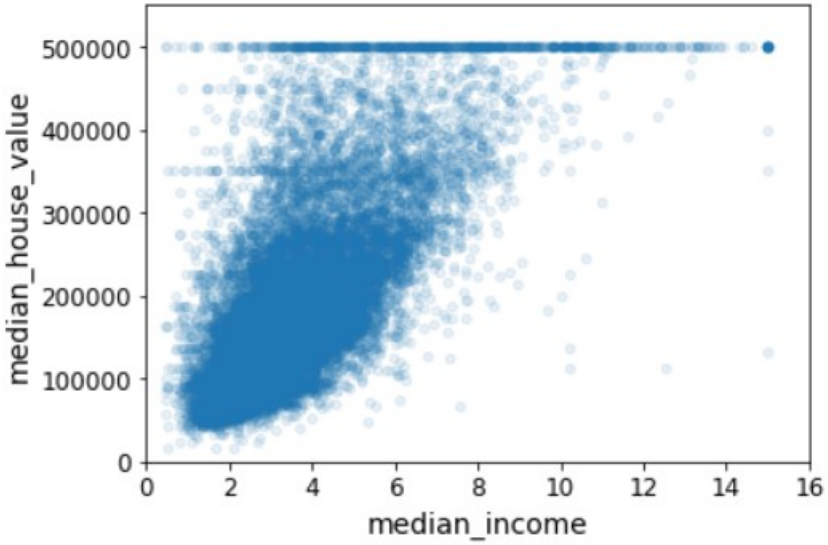
Looking deeper

- We can see a few clear trends in the data here
- First of all, the clear upward trend showing that positive correlation we saw – and it's quite strong because the points are fairly well aligned rather than spread out



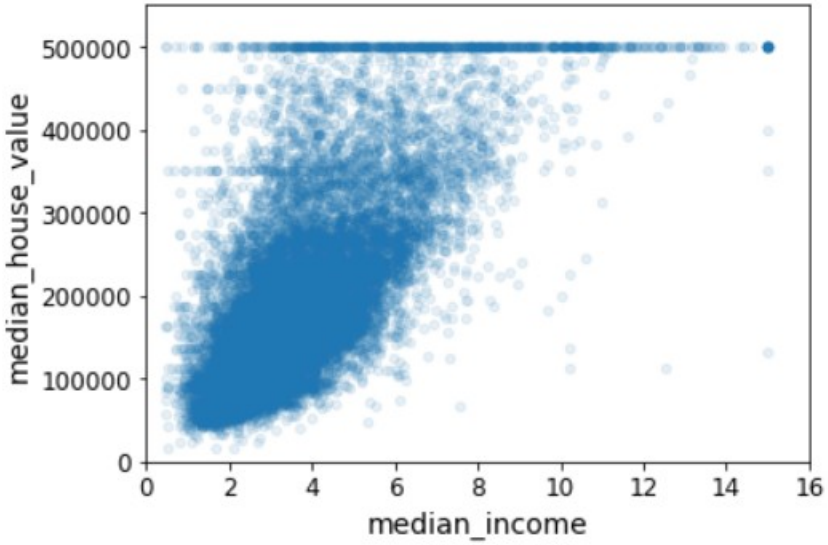
Looking deeper

- Secondly, we can see a horizontal line at the top, representing the price-capping we saw before
- Thirdly, and perhaps most interestingly, we can see several other horizontal lines, at around $450k, and $350k and $250k



Looking deeper

- We may want to try to algorithmically remove these quirks, to avoid machine learning models trying to learn to emulate them
- Particularly the top, very strong and obvious line



Feature Engineering

- Finally, before we prepare the data for machine learning we may want to consider engineering new features from our existing ones
- This mostly means combining them together in ways that make sense
- For example, the total number of rooms in a district isn’t all that useful, if you don’t know how many households there are. What you actually want is number of rooms per household.
- Similarly, the total number of bedrooms isn’t that useful by itself, you probably want to compare it to overall number of rooms.
- Population per household also seems like it could be interesting

Feature Engineering

```
housing["rooms_per_household"] = housing["total_rooms"]/housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"]/housing["total_rooms"]
housing["population_per_household"]=housing["population"]/housing["households"]
```

Feature Engineering

```
corr_matrix = housing.corr()
corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
]: median_house_value          1.000000
   median_income               0.687160
   rooms_per_household         0.146285
   total_rooms                 0.135097
   housing_median_age          0.114110
   households                  0.064506
   total_bedrooms              0.047689
   population_per_household   -0.021985
   population                 -0.026920
   longitude                  -0.047432
   latitude                   -0.142724
   bedrooms_per_room          -0.259984
   Name: median_house_value, dtype: float64
```

Looking good!

- The newly created bedrooms per room attribute is well correlated with the median house value compared to either total number of rooms, or bedrooms.
- The number of rooms per household is more informative,as the larger the houses, the more expensive they will clearly be

Data Exploration

- This is a fairly swift data exploration step
- In reality, this would merely be the initial step – once we had a good feel for the data and had tried some machine learning, we would likely revisit this step
- Other attributes may appear more promising, other things may suggest themselves on revisiting

Data Cleaning

- We will revert to a clean data set for now, we should be able to do this by copying our initial training set
-

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

- We saw in a previous session, that total_bedrooms has missing values which we need to fix, we have three options to deal with this
  - Remove the districts with missing data for total bedrooms
  - Get rid of the total bedrooms attribute, thus negating the problem that it has missing data
  - Set the missing values to some *imputed* value

Data Cleaning

- We do have functions available for all of these options

```
sample_incomplete_rows.dropna(subset=["total_bedrooms"])
```

```
sample_incomplete_rows.drop("total_bedrooms", axis=1)
```

```
median = housing["total_bedrooms"].median()
sample_incomplete_rows["total_bedrooms"].fillna(median, inplace=True)
```

Data Cleaning

- Another option than doing it manually using pandas functions is to use scikit-learns “SimpleImputer” class
-

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
```

- This can only be used on numerical attributes, so to use it you need a copy of the data without the textual attribute (ocean_proximity)

```
housing_num = housing.drop("ocean_proximity", axis=1)
```

```
imputer.fit(housing_num)
```

Data Cleaning

- This imputer is now “trained” and can be used on the training set to replace missing values with the calculated medians
-

```
X = imputer.transform(housing_num)
```

- That returns a raw numpy array (rather than a pandas dataframe) so we can load it back into a dataframe by doing the following

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing.index)
```

Tutorial

In your groups

- Try out some of these techniques on your data
- What variables are correlating well with your target value?
- What feature engineering can you do to create more variables that may be more informative?
- How do those correlate?
- Are any of the other visualisations we've looked at in the past interesting when linked to those data?
- Can you find other ways to impute values that may be more effective than the methods demonstrated?
- Remember, even if your eventual task is to train a machine learning algorithm (which it may not always be) you will still likely need a good report with some informed science and experimentation to inform this process, and allow you to justify it if necessary